Assignment 02

Signal processing, PCM, predictive coding, information theory

Disclaimer & Sources

Exercises 2, 3 and 5 were worked on in collaboration with Philipp Wolger (Group Schöffmann) during a productive hour-long afternoon coffee session. All solutions are my own but the path to reach them was discussed with my esteemed colleague and the results were compared. Exercise 5 was solved using real-time peer-programming to create a shared blueprint for the solution.

I have employed no generative AI in my solutions, and all answers are based on the lecture slides by Prof. Klaus Schöffmann unless stated otherwise.

1 Signal Processing - Analog/Digital Conversion

Explain how an analog signal can be converted into a digital signal. Pay particular attention to the following terms:

a. Sampling rate
b. Quantization

A: An analog signal refers to a continuous change or measurement of physical phenomena. However, when recording such a signal it usually has to be converted into a digital format, as recording or storing such phenomena at infinite resolution is not possible (unless assuming knowledge of the exact underlying wave). Thus, it is converted into a digital signal, where only individual signal values or measurements are represented at discrete points in time.
How often such a measurement is taken is referred to as the “sampling rate”, which is typically measured in Hz (occurrence per second). A sampling rate of 1 Hz would thus correspond to one measurement per second.
Additionally digital signals are typically stored as discrete numbers, owing to a fixed amount of space available for encoding (e.g. a certain number of bits that are available to represent some specific information). This is called “quantization” whereby an arbitrary floating-point number is mapped onto a discrete (typically the closest) value.

Explain the Nyquist-Shannon sampling theorem and the possible consequences of undersampling. What are the effects of undersampling that you are familiar with in everyday life? Why is it important to address human perception/information processing in the context of multimedia?

A: The Nyquist-Shannon sampling theorem states that for lossless digitization, the sampling rate should be at least twice the maximum frequency component occurring

in a recording. Several different frequencies may exist in a recording and the highest frequency thus determines the minimum required sampling rate.
If it is not possible to sample fast enough, undersampling occurs. Undersampling leads to aliases, which are artifacts that do not belong to the original signal. In sampling, aliases lead to an incorrect frequency that is lower than the correct one. Anti-aliasing filters may reduce the impact of this, but in most cases, information will be irretrievably lost regardless.

Addressing human perception and information processing in the context of multimedia has several advantages. For one, it determines what criteria must be met by digitized media to not be perceived as jarring or "incorrect" with respects to the original signal, as might be the case with too low framerates, undersampling or poorly chosen quantization levels. On the other hand, consideration of human perception also allows to drop information that is not relevant to it, e.g. frequencies outside of human hearing range, color perception, our limited temporal resolution in vision or ability to differentiate between very close together amplitudes and frequencies. This allows for smaller file sizes and more efficient handling of media.

2 Signal-to-Noise Ratio

Answer the following questions:

a. Explain the signal-to-noise ratio (SNR).

A: The signal-to-noise ratio (SNR) describes the ratio of power of the correct desired signal to background noise. A high signal-to-noise ratio is thus desirable as the noise will be comparatively minor with respect to the signal, whereas a low SNR would mean a much more significant impact of the noise compared to the desired signal.

b. What unit is used to express SNR?

A: SNR is expressed in decibel (dB).

c. What does an SNR of 0 dB mean?

A: SNR is calculated through the formula $SNR = 10 * \log_{10}(Signal_{Energy} / Noise_{Energy})$. SNR becomes 0 when the signal is as large as the noise, so it means that the power of the desired signal is equal to the power of the noise.

d. The signal $s(t) = A_s \cdot \sin(4 \cdot \pi \cdot f_s \cdot t)$ is transmitted over a noise-sensitive channel C. You know that the transmission adds a noise signal of the form $n(t) = A_n \cdot \sin(10 \cdot \pi \cdot f_n \cdot t)$ to the signal s(t). Calculate the SNR for
   i. $A_s = 40$, $A_n = 5$,

A:
$Signal_{Energy} = A^2 / 2 = 40^2 / 2 = 1600 / 2 = 800$
$Noise_{Energy} = A^2 / 2 = 5^2 / 2 = 25 / 2 = 12.5$

$SNR = 10 * \log_{10}(Signal_{Energy} / Noise_{Energy})$ [dB] = $10 * \log_{10}(800 / 12.5)$ =
= $10 * \log_{10}(64) = 18.06179973984... \approx$ **18.06 dB**

ii. $A_s = 50$, $A_n = 30$.

A:
$Signal_{Energy} = A^2 / 2 = 50^2 / 2 = 2500 / 2 = 1250$
$Noise_{Energy} = A^2 / 2 = 30^2 / 2 = 900 / 2 = 450$
$SNR = 10 * \log_{10}(Signal_{Energy} / Noise_{Energy})$ [dB] = $10 * \log_{10}(1250 / 450)$ =
= $10 * \log_{10}(2.777...) = 4.4369749923... \approx$ **4.44 dB**

e. Suppose $A_n = 5$. What should be the minimum A_s to maintain an SNR of at least 30 dB?

A:
$SNR = 10 * \log_{10}(Signal_{Energy} / Noise_{Energy})$ [dB]
$Noise_{Energy} = A^2 / 2 = 5^2 / 2 = 25 / 2 = 12.5$
$30 = 10 * \log_{10}(Signal_{Energy} / 12.5)$
$3 = \log_{10}(Signal_{Energy} / 12.5)$
$10^3 = Signal_{Energy} / 12.5$
$1000 * 12.5 = Signal_{Energy}$
$Signal_{Energy} = 12500$

$Signal_{Energy} = A^2 / 2$
$A = sqrt(Signal_{Energy} * 2)$
$A_s = sqrt(12500 * 2) = sqrt(25000) = 158.1138830084... \approx$ **158.114**

f. What is the signal-to-quantization noise ratio (SQNR)?

A: The signal-to-quantization-noise ratio describes the power of the signal compared to the power of the noise introduced by quantization. This quantization noise stems from quantization error, the difference between the actual value and the sampled value.

(A = amplitude, L = quantization levels, N = bits)
- $\Delta = 2A / L = 2A / 2^N$
- $SQNR = 10 * \log_{10}((2^N - 1)^2 / (1/2)^2) = 20 * \log_{10}((2^N - 1) / (1/2))$

Strictly speaking, $(1/2)^2$ in the denominator would have to be $\Delta^2/12$ but it is simplified here (as in the lecture slides).

g. Calculate the SQNR for the signal $s(t) = A_s \cdot \sin(2 \cdot \pi \cdot f_s \cdot t)$ with $A_s = 20$, if you have
i. 6,

A:
$SQNR = 20 * \log_{10}((2^N - 1) / (1/2))$
= $20 * \log_{10}((2^6 - 1) / (1/2)) = 20 * \log_{10}((64 - 1) * 2) = 20 * \log_{10}(63 * 2) = 20 * \log_{10}(126) = 20 * 2.1003705451 = 42.007410902351 \approx$ **42.01 dB**

ii. 8,

A:
SQNR = $20 * \log_{10}((2^N - 1) / (1/2))$
$= 20 * \log_{10}((2^8 - 1) / (1/2)) = 20 * \log_{10}((256 - 1) * 2) = 20 * \log_{10}(255 * 2) = 20 * \log_{10}(510) = 20 * 2.7075701761 = 54.151403522 \approx$ **54.15 dB**

iii. 10 bit

A: SQNR = $20 * \log_{10}((2^N - 1) / (1/2))$
$= 20 * \log_{10}((2^{10} - 1) / (1/2)) = 20 * \log_{10}((1024 - 1) * 2) = 20 * \log_{10}(1023 * 2) = 20 * \log_{10}(2046) = 20 * 3.3109056294 = 66.2181125875 \approx$ **66.22 dB**

available for quantization.

3 Pulse Code Modulation / μ-Law and A-Law

Explain Pulse Code Modulation (PCM).

A: Pulse Code Modulation (PCM) describes a process of digitizing an analog signal by quantizing samples recorded in a regular interval into N discrete levels, where the intensity values are uniformly quantized, i.e. the corresponding gap in intensity between two adjacent quantization levels is always the same. This means that if the amplitude of a signal doubles, the quantized value also doubles. Human audio perception, however, does not work this way (see Weber's Law, later).

Assume that the sound waves produced by human speech are in the frequency range of 50 to 10 kHz. Use the Nyquist-Shannon sampling theorem to determine the necessary sampling rate at which the analog signal (speech) should be sampled in order to reconstruct it correctly.

A: Nyquist Theorem: $f_s >= 2 * f_m$
i.e. the sampling rate should be at least twice the maximum frequency component. Since the highest frequency component in human speech would be 10 kHz here, the minimum sampling rate would have to be 20 kHz.
20 kHz >= 2 * 10 kHz

What is the bit rate of the resulting digital signal if you use PCM with a quantization value range of

a. 8 bit,

A: N = 8, f_s = 20 kHz = 20000 Hz
Bitrate = 8 * 20000 Hz = 160000 bit/s = 160 kBit/s

b. 16 bit, and

A: N = 16, f_s = 20 kHz = 20000 Hz
Bitrate = 16 * 20000 Hz = 320000 bit/s = 320 kBit/s

c. 24 bit?

A: N = 24, f_s = 20 kHz = 20000 Hz
Bitrate = 24 * 20000 Hz = 480000 bit/s = 480 kBit/s

PCM uses linear quantization. Alternatively, you could use non-linear quantization. Explain what nonlinear quantization is and why it can be advantageous (Weber's Law).

A: Humans tend to only perceive a difference between two stimuli when a certain threshold of difference is passed. This threshold is relative to the initial stimulus, i.e. if a sound is only perceived as louder as another if it is at least 5% louder. This is known as Weber's Law and it is advantageous so that quantization can be performed more intelligently, preserving more information where humans are more sensitive and preserving less where humans are less sensitive and are unlikely to perceive a difference.
This can be achieved through non-linear quantization, where the corresponding intensity between two adjacent quantization levels is not always the same for different levels. To take advantage of Weber's law, low sound intensities should map to quantization indices much more granularly (or expressed the other way around: low quantization indices should have smaller gaps in corresponding intensities), since humans can more easily perceive differences between them, while there may be much bigger gaps in intensities for quantization levels mapping to higher sound intensities, as humans will not perceive these differences as strongly.

μ-Law and A-Law allow for nonlinear quantization. Outline how these techniques can be used in conjunction with PCM to encode an analog input signal into a digital one. What changes should be made to PCM if μ-Law/A-Law is used?

A: μ-Law and A-Law are both approaches to still use standard PCM while taking advantage non-linear quantization (despite PCM using uniform quantization). First, the analog signal is transformed from the "raw space" into a "theoretical space" through a function, which increases the difference of quieter signals and decreases the difference of louder signals so that when signals in the theoretical space are linearly quantized, less intense signals are more accurately preserved, as humans will hear more difference there. μ-Law is one such common transformation, A-law is a very similar transformation used in telephony in Europe.

Encoding requires the steps detailed above before PCM is applied, and vice versa, the decoder must convert the values corresponding to the quantization levels back from the theoretical space into the "raw space" so that the output matches the original signal.

4 Predictive Coding / DPCM

Explain the term Predictive Coding. Explain the term Differential Pulse Code Modulation (DPCM) and find out what is the relationship between DPCM, PCM and Predictive Coding.

A: Standard PCM encodes every sample individually which is inefficient, considering the fact that in most cases, differences between two adjacent samples are small. Differential Coding instead examines the differences between samples. Since there is usually some consistency, most differences to the prior sample will be small, and short codewords can be used to represent common values while longer codewords are used to encode rare values, which will rarely outweigh the benefits of the size reduction through the shortening of frequent values.

Predictive coding takes differential coding one step further and introduces a predictor which will deterministically predict the next sample based on previous samples. Since both decoder and encoder share the same predictor, only the difference from the expected value has to be stored, which can reduce needed storage even further by reducing the amount of values that differ from the most common ones (as the predictor should be near-correct and offsets from it should thus mostly be in the same small range centering aroud zero).

Differential Pulse Code Modulation (DPCM) is the same as predictive coding with the difference being that the error is also quantized (which wasn't the case previously, but this is impractical in digitzation). Predictions are thus based on the reconstructed signal (both for the encoder and decoder), i.e. the prediction + the quantized error.

Assume that the given DPCM scheme uses the following variable labels:

- let f be an integer sequence of input signals.
- let f_n be the nth (actual) value in the sequence
- $\hat{f}_n$ is the nth predicted value of the sequence
- e_n is the nth error between f_n and $\hat{f}_n$
- let $\tilde{e}_n$ be the nth quantized error
- $\tilde{f}_n$ is the nth reconstructed value.

In the given scheme, the values of the variables are obtained from the following equations:

- $\hat{f}_n = \text{trunc}[(\tilde{f}_{n-1} + \tilde{f}_{n-2}) / 2]$
- $e_n = f_n - \hat{f}_n$
- $\tilde{e}_n = Q[e_n] = 16 \cdot \text{trunc}[(255+e_n) / 16] - 256 + 8$

transmit codeword ($\tilde{e}_n$)

reconstruct: $\tilde{f}_n = \hat{f}_n + \tilde{e}_n$

Calculate which codewords are produced when the above DPCM scheme is used and the input is the sequence: ($f_1 = 126$, $f_2 = 176$, $f_3 = 161$, $f_4 = 107$, $f_5 = 128$). Calculate the values to be transferred and reconstruct them all, afterwards. Explain what the quantization function Q is for?

Note(s):

- The function "trunc" generates an integer by discarding the decimal places of a floating point number.
- Duplicate the first value of the sequence before starting to generate the DPCM code (values $\tilde{f}_{n-1}$ and $\tilde{f}_{n-2}$) and initialize the quantized error $\tilde{e}_n$ for n=1 with 0. This way you achieve that the first reconstructed value is not corrupted.

A:
Encoding:
$f_1 = 126$
$\tilde{f}_{-1} = 126$, $\tilde{f}_0 = 126$ ->
$\hat{f}_1 = 126$
$e_1 = 0$
$\tilde{e}_1 = 16 * \text{trunc}(255/16) - 256 + 8 = 16 * 15 - 256 + 8 = -8$ (Override: 0)
$\tilde{f}_1 = \hat{f}_1 + \tilde{e}_1 = 126 - 0 = 126$

$f_2 = 176$
$\tilde{f}_0 = 126$, $\tilde{f}_1 = 126$ ->
$\hat{f}_2 = \text{trunc}((126+126)/2) = 126$
$e_2 = 176 - 126 = 50$
$\tilde{e}_1 = 16 * \text{trunc}((255+50)/16) - 256 + 8 = 16 * 16 - 256 + 8 = 56$
$\tilde{f}_2 = \hat{f}_2 + \tilde{e}_2 = 126 + 56 = 182$

$f_3 = 161$
$\tilde{f}_1 = 126$, $\tilde{f}_2 = 178$ ->
$\hat{f}_3 = \text{trunc}((126+178)/2) = 152$
$e_3 = 161 - 152 = 9$
$\tilde{e}_3 = 16 * \text{trunc}((255+9)/16) - 256 + 8 = 16 * 19 - 256 + 8 = 8$
$\tilde{f}_3 = \hat{f}_3 + \tilde{e}_3 = 152 + 8 = 160$

$f_4 = 107$
$\tilde{f}_2 = 182$, $\tilde{f}_3 = 160$ ->
$\hat{f}_4 = \text{trunc}((182+160)/2) = 171$
$e_4 = 107 - 171 = -64$
$\tilde{e}_4 = 16 * \text{trunc}((255-64)/16) - 256 + 8 = 16 * 11 - 256 + 8 = -72$
$\tilde{f}_4 = \hat{f}_4 + \tilde{e}_4 = 171 - 72 = 99$

$f_5 = 128$
$\tilde{f}_3 = 160$, $\tilde{f}_4 = 99$ ->
$\hat{f}_5 = \text{trunc}((160+99)/2) = 129$
$e_5 = 128 - 129 = -1$
$\tilde{e}_5 = 16 * \text{trunc}((255-1)/16) - 256 + 8 = 16 * 15 - 256 + 8 = -8$

$\tilde{f}_5 = \hat{f}_5 + \tilde{e}_5 = 129 - 8 = 121$

Encoded: 126 56 8 -72 -8

Decoding:
$\tilde{f}_1 = 126$

$\hat{f}_2 = \text{trunc}((\tilde{f}_0 + \tilde{f}_1) / 2) = \text{trunc}((126+126) / 2) = 126$
$\tilde{f}_2 = 126 + 56 = 182$

$\hat{f}_3 = \text{trunc}((\tilde{f}_1 + \tilde{f}_2) / 2) = \text{trunc}((126+182) / 2) = 154$
$\tilde{f}_3 = 154 + 8 = 162$

$\hat{f}_4 = \text{trunc}((\tilde{f}_2 + \tilde{f}_3) / 2) = \text{trunc}((182+162) / 2) = 172$
$\tilde{f}_4 = 172 - 72 = 100$

$\hat{f}_5 = \text{trunc}((\tilde{f}_3 + \tilde{f}_4) / 2) = \text{trunc}((162+100) / 2) = 131$
$\tilde{f}_5 = 131 - 8 = 123$

Decoded 126 182 162 100 123

The quantization function Q maps arbitrary values to values corresponding to discrete steps. Here, the function starts at 8 and every step increases/decreases it by 16.

5 DPCM Implementation

In Moodle you can find the Python project DPCM. It consists of the file `dpcm.py` and the input file `sequence.csv`. Download both files into the same folder and open the project in your development environment. You can run the program with `python dpcm.py`.

Your task is to implement the DPCM method. The basic structure for the implementation is already given in the file `dpcm.py`. The following functions have to be implemented:

- `encode(values)`
  This function encodes a sequence of numbers (list of int) using DPCM and returns the encoded sequence (list of int). The same scheme as in Exercise 4 applies to the implementation. Also refer to the notes from Exercise 4. Make sure that the generated number sequence contains only that information which is actually necessary for a reconstruction.
- `decode(encoded_values)`
  This function decodes a DPCM encoded number sequence and returns the reconstructed values (list of int).
- `distortion(recon_values, orig_values)`

This function calculates the distortion of the decoded DPCM number sequence compared to the original input sequence and returns it as a float. Distortion is defined as $\Sigma(n=1, N): (\tilde{f}_n - f_n)^2 / N$.

If implemented correctly, you will get the following output on the console:

```
Input: 126 176 161 107 128 130 150 140 205 230 232 238 250 230 220
210 207 200 100

Encoded: 126 56 8 -72 -8 24 24 -8 56 56 24 8 8 -8 -24 -8 -8 -8 -104

Decoded: 126 182 162 100 123 135 153 136 200 224 236 238 245 233 215
216 207 203 101

Distortion: 18.05
```

Note(s):

- The first value of the encoded sequence is the original value f_1, since the decoder needs it to start the reconstruction. All following values are the quantized errors $\tilde{e}_n$.
- Python's integer division // rounds towards negative infinity, whereas "trunc" rounds towards zero. For the values occurring in this exercise, both yield the same result.
- You can test your implementation even before all functions are implemented. After implementing encode(..), line 2 of the output should already match the output from above.

A: For solution, see attached Python project

6 DPCM Implementation - Questions

Answer the following questions to the DPCM implementation in Excercise 5:

1. How many different values can the quantized error $\tilde{e}_n$ take? How many bits per value would be needed with a fixed-length code? Compare this to the number of bits needed for e_n without quantization and for the original values f_n.

   A: In this implementation, for lack of any bounding or modulo operator anywhere within the code, the quantized error could theoretically take infinitely many values. However, given that the error values can only assume values in [-255, 255] as should be the case from the specification, we can make a meaningful statement. Given this input range, the output for the quantized error ranges from the lowest possibility -248 to the highest +248. This means that there are 496 values in between but because of the division by 16 in combination with the trunc-function, the result can be one of 32 discrete values. As a consequence, only 5 bits are required, as these suffice to encode 32 numbers (0 to 31). Without quantization, the error could be any value within

[-255, 255], so 9 bits would be required to allow encoding all of the 512 (actually 511) possibilities. If only the original values were encoded, 8 bits would be required to encode all 256 possibilities from the range [0, 255].

2. The encoded sequence contains values such as −104 or 56. Does transmitting $\tilde{e}_n$ require more bits than transmitting a quantization index $k = \text{trunc}[(255+e_n) / 16]$ instead? How would the decoder obtain $\tilde{e}_n$ from k?

A: Yes, in general transmitting the entire quantized error requires more bits, since when treating the values naively, they may be from the range [-248, 248], as established in the previous question. Treating this as a number would require 9 bits.
When transmitting the quantization index instead, it may only be from range [0, 31], which means that 5 bits suffice. The quantized error can be obtained from the decoder by simply multiplying the received quantization level k by 16 and performing the rest of the calculation (- 256 + 8), as the formula remains static and the variable part of the formula has been transmitted (including the trunc-calculation).

3. Compare the total number of bits needed to transmit the given input sequence (a) with PCM (8 bits per value) and (b) with your DPCM implementation using a fixed-length code for the quantized errors. Why could Huffman coding of $\tilde{e}_n$ reduce this number even further?

A: The given input sequence consists of 19 samples. With PCM (8 bits per value), we would need 19 * 8 = 152 bits. Using my DPCM implementation and a fixed-length code for the quantized errors, 5 bits would suffice (as shown in the previous exercise), thus we would need 19 * 5 = 95 bits.
With Huffman coding, we would use a variable-length code which would use the shortest codes for the most common values and the longest codes for the most uncommon ones. Given that we accurately know the probability distribution of values, Huffman coding provably produces an optimal code that yields the shortest possible encodings within our domain (see slides for Introduction to Computing Science by Prof. Andreas Bollin).